Reviewer Pack — Hyperbolic Geometry of Discrete Groups and Monotone Circuit …

Entry a34349c5d777 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-21 14:21:39 UTC

Hyperbolic Geometry of Discrete Groups and Monotone Circuit Lower Bounds

Entry ID: `a34349c5d777` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-21 14:21:39 UTC

1. Conjecture

Field A (mathematical branch): Hyperbolic Geometry **Field B** (complexity object): Monotone Boolean Circuit Complexity

Statement:

> ['For any hyperbolic surface Σ with genus g, there exists a monomial degree D such that any monotone boolean circuit of size S(n) solving the satisfiability problem on Σ with n variables has size at most D^g * S(n).', 'For all sufficiently large n, this bound holds with D = 2.', 'This bound does not hold for a hyperbolic surface Σ with genus g ≥ 4.']

Rationale (proposer's reasoning):

> ['Hyperbolic geometry is known to exhibit unique properties that can lead to strong lower bounds on computational complexity. The discrete groups acting on these surfaces can be used to construct gadgets in monotone circuits, which may lead to new types of lower bounds.', 'The hyperbolic nature of the surface introduces exponential growth in the number of possible configurations, which could potentially lead to a polynomial relationship between the size of the circuit and the genus of the surface.']

Taxonomy category: `HYPERBOLIC_GEOemetRY_MONOTONE_CIRCUIT` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `955f07492c8ef70e`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For a hyperbolic surface Σ with genus g, the monotone circuit size S(n) will be compared to D^g * S(n) where D = 2 for all n tested. A result is considered supported if the ratio |D^g * S(n)/S(n)| ≤ 1.05 across at least 10 seeds and a falsification condition occurs when any seed exceeds this ratio by more than 5%.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.70	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_hyperbolic_surface(g):
        if g < 4:
            # Placeholder for actual hyperbolic surface generation logic
            return "hyperbolic_surface"
        else:
            raise NotImplementedError("Mapping undefined for genus >= 4")

    def construct_satisfiability_instance(surface, n):
        # Placeholder for constructing satisfiability instance on the surface
        return "satisfiability_instance"

    def measure_circuit_size(instance):
        # Placeholder for measuring circuit size to solve the instance
        return random.randint(10, 100)

    g_values = [5, 10, 15, 20, 30, 40]
    total_size = 0
    instances_tested = 0

    for g in g_values:
        try:
            surface = generate_hyperbolic_surface(g)
            instance = construct_satisfiability_instance(surface, n=10)  # Fixed n to avoid sub-as
            circuit_size = measure_circuit_size(instance)
            total_size += circuit_size
            instances_tested += 1
```

```
        except NotImplementedError:
            return {
                "metric_name": "circuit_size",
                "metric_value": None,
                "instances_tested": instances_tested,
                "conjecture_holds": False,
                "counterexample": "mapping_undefined"
            }

    if instances_tested == 0:
        return {
            "metric_name": "circuit_size",
            "metric_value": None,
            "instances_tested": instances_tested,
            "conjecture_holds": False,
            "counterexample": "no_instances_generated"
        }

    mean_size = total_size / instances_tested
    conjecture_holds = all(abs(2**g * mean_size - mean_size) <= 1.05 * mean_size for g in g_values
    counterexample = "" if conjecture_holds else f"Genus {g_values[0]} does not satisfy the bound'

    return {
        "metric_name": "circuit_size",
        "metric_value": mean_size,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_value = sum(result["metric_value"] for result in results if result["metric_value"] is not
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction={support_fraction}")
    elif any(abs(result["metric_value"] - 2**g * result["metric_value"]) > 1.05 * result["metric_v
        first_failing_seed = next(seed for seed, result in enumerate(results) if abs(result["metri
        print(f"RESULT: FALSIFIED counterexample='genus {g} does not satisfy the bound' first_fail
    else:
        print("RESULT: INCONCLUSIVE")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
{'metric_name': 'circuit_size', 'metric_value': None, 'instances_tested': 0, 'conjecture_holds': Fal
TRIAL: {'metric_name': 'circuit_size', 'metric_value': None, 'instances_tested': 0, 'conjecture_hold
TRIAL: {'metric_name': 'circuit_size', 'metric_value': None, 'instances_tested': 0, 'conjecture_hold
TRIAL: {'metric_name': 'circuit_size', 'metric_value': None, 'instances_tested': 0, 'conjecture_hold
TRIAL: {'metric_name': 'circuit_size', 'metric_value': None, 'instances_tested': 0, 'conjecture_hold
TRIAL: {'metric_name': 'circuit_size', 'metric_value': None, 'instances_tested': 0, 'conjecture_hold
TRIAL: {'metric_name': 'circuit_size', 'metric_value': None, 'instances_tested': 0, 'conjecture_hold
TRIAL: {'metric_name': 'circuit_size', 'metric_value': None, 'instances_tested': 0, 'conjecture_hold

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_46173129.py", line 91, in <module>
    elif any(abs(result["metric_value"] - 2**g * result["metric_value"]) > 1.05 * result["metric_value
         ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_46173129.py", line 91, in <genexpr>
    elif any(abs(result["metric_value"] - 2**g * result["metric_value"]) > 1.05 * result["metric_value
                                                  ~~~~~^~~~~~~~~~~~~~~~~~~~~~~
TypeError: unsupported operand type(s) for *: 'int' and 'NoneType'
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test code crashed before producing data, which prevents us from verifying the conjecture's validity. | next: Review and debug the test code to ensure it can run without crashing and produce results.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15018
2	preregistration	ollama_remote	glm4:latest	0	0	16615
3	novelty	ollama_remote	glm4:latest	0	0	8982
4	novelty	ollama_remote	glm4:latest	0	0	8201
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18688
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11535
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13605
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9950
9	judge	ollama_remote	glm4:latest	0	0	11426

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 114019 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/a34349c5d777.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/a34349c5d777.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/a34349c5d777.tar.gz` (if generated)